

OpenVAF-r Compiler Internals

A ground-up guide to how the openvaf-r Verilog-A compiler works inside

Ngspice + OpenVAF Enhancements project

July 2026

Contents

OpenVAF-r Compiler Internals	3
How to read this document	4
Chapter 1 — What OpenVAF-r is, and the problem it solves	5
Chapters 4–13 are those seven jobs, in order. But first, the bird's-eye view.	8
Chapter 2 — The whole pipeline at a glance	8
Each lowering <i>throws away</i> information the next stage doesn't need. By the time we reach MIR, the code no longer knows the difference between a user variable and a temporary — it is just a graph of arithmetic. That is precisely what makes automatic differentiation and optimisation tractable.	11
Chapter 3 — Background you'll need	11
With these four ideas — the DAE system, SSA, automatic differentiation, and the query database — you have the scaffolding. Now we walk the pipeline.	14
Chapter 4 — Stage 1: reading and preprocessing the source	14
At the end of Stage 1 we have a flat, fully-expanded stream of <code>SyntaxKind</code> tokens, each still traceable to its origin. That stream feeds the parser.	16
Chapter 5 — Stage 2: parsing into a syntax tree	16
The product of Stage 2 is a CST for the whole file, with a typed AST overlay, still fully connected to source spans through the source map. Crucially, the tree is <i>purely syntactic</i> : it knows that $V(p, n)$ is a call-like expression with two arguments, but it does not yet know that V means "the potential of a branch", that p and n are electrical nodes, or that the expression's type is <code>real</code> . Establishing all of that meaning is the job of the HIR.	17
Chapter 6 — Stage 3: the HIR (meaning, names, and types)	17
What the HIR is <i>not</i> is something you can differentiate or optimise. It is still tree-shaped, still tied to language concepts, still carrying control flow as nested statements. To do numerical work we lower it into the MIR — and, at the same time, discover the device's DAE topology. That double act is the subject of the next two chapters.	19
Chapter 7 — Stage 4: lowering HIR to MIR	19
Lowering a module produces (roughly): one MIR Function for the evaluation (the analog block), companion functions for parameter initialisation and model/instance setup, and the fully populated <code>HirInterpreter</code> that says what every input and output means. At this point the code is pure SSA dataflow with clearly labelled boundary values — ready both for the <i>topology analysis</i> that builds the DAE (Chapter 9) and for the <i>differentiation</i> that builds the Jacobian (Chapter 10). First, though, let's look at what the MIR actually is.	21
Chapter 8 — The MIR in detail (SSA, dataflow, opcodes)	21
The textual form (produced by <code>openvaf/mir/src/write.rs</code>) looks like the SSA sketch in §3.2: <code>blockN: labels, one vN = opcode args per line, ending each block with br/jmp/exit.</code> We use this in the worked example (Chapter 14) to see a real device's residual and its machine-generated derivatives side by side.	24
Chapter 9 — Stage 4 (cont.): building the DAE system (<code>sim_back</code>)	24

At the end of Stage 4 we have, per module, a complete mathematical description of the device: what it solves for, what it contributes, and how everything derives — but the derivatives themselves are still only <i>structural</i> (we know <i>which</i> Jacobian entries exist). Computing the derivative <i>values</i> is automatic differentiation, and it happens as the back end lowers each of these into concrete MIR functions.	26
Chapter 10 — Stage 6: automatic differentiation (the Jacobian)	26
After autodiff, the module's MIR contains both the residual computations <i>and</i> all live Jacobian computations, as one big SSA graph with lots of shared subexpressions (the whole point of doing AD on SSA). That graph is redundant in places — the same product may be computed for the value and again inside a derivative — which is exactly what the optimiser cleans up next.	28
Chapter 11 — Stage 7: optimising the MIR	28
Chapter 12 — Stage 8: code generation with LLVM	30
The object files are handed back to <code>compile()</code> , which links them (Chapter 13) into the final <code>.osdi</code>	32
Chapter 13 — Stage 9: the OSDI descriptor and linking	32
The descriptor carries the OSDI ABI version. This repository's compiler emits the reloaded descriptor (base 0.4, extended through 0.7), and a host negotiates against it. If you compile with a stale binary that only writes v0.4 base entry points and load it into a host expecting the ≥ 0.7 layout, the host rejects it with a clear version error — the exact failure you hit when mixing compiler and simulator generations. The extensions between 0.4 and 0.7 are additive descriptor fields (a longer descriptor whose prefix stays 0.3/0.4-compatible, which is why <code>OSDI_DESCRIPTOR_SIZE</code> is exported for hosts that stride the descriptor array manually).	34
Chapter 14 — A complete worked example: <code>resistor.va</code> end to end	34
The compiler carried enough <i>semantic</i> knowledge (the module's name) and enough <i>domain</i> knowledge (a table of reserved simulator device names) to warn about a footgun that has nothing to do with the Verilog-A language itself — a good example of the value the HIR layer adds over raw parsing.	37
Chapter 15 — Cross-cutting machinery: database, diagnostics, tests	37
Fast tests run by default; <code>RUN_SLOW_TESTS=1</code> adds the heavy models. This layered test strategy (per-stage snapshots + end-to-end integration + an interpreter oracle) is what makes it feasible to add a language feature or fix a numeric law and be confident nothing else moved.	39
Chapter 16 — How a simulator uses the <code>.osdi</code> at runtime	39
Every design choice in the descriptor traces back to this loop being <i>hot</i> : parameters are pre-processed in <code>setup</code> so <code>eval</code> does no defaulting; Jacobian slots are pre-wired so <code>load</code> does no matrix search; <code>eval</code> and <code>load</code> are separate so <code>compute</code> isn't repeated per stamp target; the whole thing is native machine code with a flag-gated fast path. The compiler front end exists to <i>understand</i> the model; the back end and OSDI layer exist to make this loop as cheap as physically possible.	40
Chapter 17 — Reference appendices	40
Closing note	44

OpenVAF-r Compiler Internals

A ground-up, no-prior-knowledge guide to how the OpenVAF-reloaded Verilog-A compiler works inside.

How to read this document

This document explains, from the very beginning, how the **openvaf-r** compiler turns a Verilog-A device model into a machine-code plug-in that a circuit simulator can run. It is written for someone who has *never* built a compiler and may not know what a “circuit simulator” needs from a device model. Every term is introduced before it is used.

The document is one long file, organised into numbered chapters. You can read it straight through, or jump to a chapter using the table of contents. The recommended path:

- Chapters 1–3 give you the mental model: what the compiler is *for*, the shape of the whole pipeline, and the handful of background ideas (circuit equations, SSA form, incremental databases) that the rest of the document leans on. Read these first.
- Chapters 4–13 walk the pipeline one stage at a time, in the order the compiler actually runs them. Each chapter names the exact Rust crate(s) responsible, shows the key data structures with real code, and gives a small worked example.
- Chapter 14 ties it together by tracing one tiny model (`resistor.va`) all the way from source text to a loaded `.osdi` library.
- Chapters 15–17 cover the cross-cutting machinery (the database, diagnostics, tests) and reference material (crate map, opcode table, glossary, debugging flags, “how do I add X”).

Throughout, file paths are given relative to the **openvaf-r** source tree (the `OpenVAF-master/` directory), e.g. `openvaf/mir/src/lib.rs`. Code shown in boxes is either copied from that tree or is a deliberately simplified sketch — sketches are always labelled as such.

A note on names. The project is *OpenVAF-reloaded*; its binary is `openvaf-r`. It is a community fork of Pascal Kuthe's original *OpenVAF*, maintained by Árpád Bűrmen, with LLVM 18 support added by Kreijstal. When this document says “OpenVAF” it means the reloaded compiler in this repository.

Chapter 1 — What OpenVAF-r is, and the problem it solves

1.1 The one-sentence summary

OpenVAF-r is a compiler that reads a Verilog-A file describing an electronic device and writes a `.osdi` shared library containing optimised native code that a SPICE-class circuit simulator loads at runtime to simulate that device.

To understand why that is useful — and why it is *hard* — you need three pieces of background: what a circuit simulator does, what a “compact model” is, and what Verilog-A is.

1.2 What a circuit simulator does (the 3-minute version)

A circuit simulator such as ngspice answers questions like “*if I build this circuit, what voltage appears at this node over time?*” It does so numerically. The circuit is turned into a system of equations based on two physical laws:

- Kirchhoff's Current Law (KCL): at every node (wire junction) in the circuit, the currents flowing in must equal the currents flowing out — the net current is zero.
- Element laws: each component says how the current through it relates to the voltages on its terminals. A resistor obeys Ohm's law, $I = V/R$. A capacitor obeys $I = C \cdot dV/dt$. A transistor obeys something far more complicated.

The simulator collects one KCL equation per node and asks: *what set of node voltages makes every node's net current zero simultaneously?* For anything beyond a trivial linear circuit, that is a system of non-linear equations, and the simulator solves it with Newton's method: guess the voltages, compute how wrong each equation is (the *residual*), compute the *slope* of each equation with respect to each voltage (the Jacobian matrix), solve a linear system to get a correction, update the guess, and repeat until the residuals are tiny.

So, concretely, for every device in the circuit, at every Newton iteration, at every time point, the simulator needs the device to tell it two things:

1. The residual contributions — how much current this device pushes into each of its terminal nodes, given the present terminal voltages (and, for dynamic behaviour, the charges whose time-derivatives are currents).
2. The Jacobian contributions — the partial derivative of each of those currents with respect to each terminal voltage.

That second requirement is the crux. The simulator needs exact derivatives, and it needs them fast, for models that can be thousands of lines of physics.

1.3 What a “compact model” is

A compact model is a set of equations that predicts a device's terminal currents and charges from its terminal voltages and a list of parameters (geometry, temperature, process constants). “Compact” means it is a closed-form approximation of the underlying semiconductor physics that is cheap enough to evaluate millions of times — as opposed to a full physical device simulation.

Industry-standard MOSFET and bipolar models (BSIM, PSP, HICUM, MEXTRAM, EKV, ...) are compact models. A modern one such as BSIM-CMG is thousands of lines of dense equations. Historically each simulator hard-coded each model in C, and every new model or fix meant patching and recompiling the simulator itself — slow, error-prone, and locking models to specific simulators.

1.4 What Verilog-A is

Verilog-A is a standard hardware-description language (a subset of Verilog-AMS, defined by the Accellera *Verilog-AMS Language Reference Manual*, "LRM") designed for writing analog/compact models *portably*. Instead of writing the device in a simulator's C internals, you write it once in Verilog-A, and any Verilog-A-aware tool can use it.

A minimal Verilog-A model of a resistor looks like this:

```
`include "disciplines.vams"

module resistor(p, n);          // two terminals, p and n
    inout p, n;
    electrical p, n;           // both terminals carry an electrical "discipline"
    parameter real R = 1000.0; // a parameter, default 1 kΩ

    analog begin
        // Contribute a current from p to n equal to V(p,n)/R (Ohm's law)
        I(p, n) <+ V(p, n) / R;
    end
endmodule
```

The key line is the contribution $I(p, n) <+ V(p, n)/R$. It means "add to the current flowing through the branch from p to n the amount $V(p, n)/R$ ". $V(p, n)$ is the voltage across the branch — a value the *simulator* supplies. The model never computes voltages; it computes currents (and charges) as functions of voltages, exactly the residual the simulator asked for in §1.2. Notice the model does not write any derivatives. Ohm's law's derivative with respect to V is obviously $1/R$, but for a real transistor the derivatives are monstrous. Producing them is the compiler's job (Chapter 10).

1.5 What OSDI is, and why compile to it

OSDI ("Open Source Device Interface") is a small binary contract — an application binary interface (ABI) — between a compiled device model and a circuit simulator. A `.osdi` file is just a normal platform shared library (`.so/.dylib/.dll` under a different extension) that exports a fixed set of C-callable functions and a descriptor table telling the simulator:

- the device's name, its terminals, and its parameters (with types, defaults, units, and bounds);
- how big its per-instance and per-model memory blocks are, and where each field lives;
- function pointers the simulator calls: **setup_model**, **setup_instance**, **eval** (compute residual + Jacobian for the present inputs), **load_jacobian/load_residual** (stamp those numbers into the simulator's matrix), plus noise and small-signal entry points.

Because OSDI is simulator-agnostic, one compiled `.osdi` runs unmodified in ngspice, SPICE OPUS, VACASK, or any other OSDI host. OpenVAF-r targets OSDI version 0.4 as its base ABI; the "reloaded" fork extends the descriptor with newer entry points (noise stride, AC stimulus, final-step callbacks) that this repository's ngspice negotiates up to v0.7. (When a mismatched-version `.osdi` is loaded, the simulator refuses it — this is exactly the "uses v0.4 / supports ≥ 0.7 " error you hit if you mix a stale compiler with a new ngspice.)

1.6 So what does the compiler actually have to do?

Given the resistor above, `openvaf-r` must:

1. Read and understand the text: strip comments, expand ``include/`define` macros, tokenise, parse into a tree, resolve names (p , n , R , V , I), and check types (is R a real? is $V(p, n)$ used as a number?).

2. Figure out the device topology: p and n are terminals; there is one branch (p, n) ; the module contributes current to it. Therefore the DAE unknowns are the node voltages at p and n , and this device stamps into the KCL equations of those two nodes.
3. Translate the **analog** block into a pure numerical function `eval(voltages, params) -> (currents, charges)` in an internal SSA form that is easy to analyse and optimise.
4. Differentiate that function automatically to get every $\partial I / \partial V$ the Jacobian needs — symbolically exact, computed by the compiler, not by finite differences.
5. Optimise the resulting code (fold constants, remove dead computations, share common subexpressions, drop Jacobian entries that are structurally zero).
6. Generate native machine code via LLVM for the `eval/load/setup` functions.
7. Emit the OSDI descriptor — the metadata table plus memory-layout information — and link everything into a single `.osdi` shared library.

Chapters 4–13 are those seven jobs, in order. But first, the bird's-eye view.

Chapter 2 — The whole pipeline at a glance

2.1 The stages, as crates

OpenVAF-r is a Rust *workspace*: a collection of small libraries (“crates”) that each own one job. The compiler is deliberately built like `rustc` — many narrow stages, each transforming one intermediate representation (IR) into the next. Here is the entire flow, with the crate that owns each step:

```
resistor.va (Verilog-A source text)
  |
  ↓   openvaf/vfs, openvaf/preprocessor, openvaf/lexer, openvaf/tokens
[ 1 ] Read & preprocess -- expand `include / `define, strip comments, tokenize
  |
  ↓   openvaf/parser, openvaf/syntax
[ 2 ] Parse             -- build a Concrete Syntax Tree (CST), then a typed AST
  |
  ↓   openvaf/hir_def, openvaf/hir_ty, openvaf/hir
[ 3 ] HIR               -- resolve names, check types (the "semantic" model)
  |
  ↓   openvaf/hir_lower + openvaf/sim_back
[ 4 ] Lower to MIR      -- turn the analog block into SSA; find the DAE topology
  |
  ↓   openvaf/mir, openvaf/mir_build
[ 5 ] MIR (SSA)         -- an untyped, optimisable control-flow graph
  |
  ↓   openvaf/mir_autodiff
[ 6 ] Auto-differentiate -- build every Jacobian derivative symbolically
  |
  ↓   openvaf/mir_opt
[ 7 ] Optimise          -- constant folding, GVN/CSE, dead-code & sparsity
  |
  ↓   openvaf/mir_llvm (uses LLVM 18)
[ 8 ] Codegen           -- MIR → LLVM IR → native object files
  |
  ↓   openvaf/osdi, openvaf/linker
[ 9 ] OSDI + link       -- emit descriptor + memory layout, link into a library
  |
  ↓
resistor.osdi (a native shared library the simulator loads)
```

Two more crates sit *underneath* everything as infrastructure:

- **openvaf/basedb** and **openvaf/hir (CompilationDB)** — the incremental database (built on `salsa`) that caches the result of each query so nothing is recomputed unnecessarily (Chapter 15).
- **lib/...** — small utility crates (`arena`, `bitset`, `typed_indexmap`, `stdx`, ...) used everywhere.

2.2 The `compile()` function — the pipeline in ~30 lines

The whole thing is orchestrated by one function, `compile()`, in `openvaf/openvaf/src/lib.rs`. Reading it is the fastest way to see the skeleton. Lightly trimmed:

```
pub fn compile(opts: &Opts) -> Result<CompilationTermination> {
    // Resolve the input path and open the incremental database.
```

```

// Creating the DB already lexes, preprocesses and parses the root file
// (lazily – the work happens the first time each result is queried).
let input = AbsPathBuf::assert(opts.input.canonicalize()?);
let db = CompilationDB::new_fs(input, &opts.include, &opts.defines,
    ↪ &opts.lints?);

// Decide where the .osdi goes (a fixed path, or a content-hashed cache file).
let lib_file = /* ... from opts.output ... */;

// STAGE 3–4a: AST → HIR → device topology. `collect_modules` walks every
// `module` in the file, type-checks it, and builds its DAE system.
// If any fatal diagnostic was produced, stop here.
let modules = match collect_modules(&db, false, &mut ConsoleSink::new(&db)) {
    Some(modules) => modules,
    None => return Ok(CompilationTermination::FatalDiagnostic),
};

// Set up the LLVM backend for the requested target machine.
let back = LLVMBackend::new(&opts.codegen_opts, &opts.target,
    ↪ opts.target_cpu.clone(), &[]);

// STAGE 4b–9: HIR → MIR → autodiff → optimise → LLVM → object files,
// plus the OSDI descriptor for each module. Returns the object-file paths.
let (paths, compiled_modules, literals) = osdi::compile(
    &db, &modules, &lib_file, &opts.target, &back, true, opts.opt_lvl,
    opts.dump_mir, opts.dump_unopt_mir, opts.dump_ir, opts.dump_unopt_ir,
);

// STAGE 9: link all object files into the final shared library, then
// delete the intermediate .o files.
link(None, &opts.target, lib_file.as_ref(), |linker| {
    for path in &paths { linker.add_object(path); }
})?;
for obj_file in paths { remove_file(obj_file)?; }

Ok(CompilationTermination::Compiled { lib_file })
}

```

Notice how little glue there is. Two calls do almost all the work:

- **collect_modules(&db, ...)** (in `sim_back`) runs the *front end*: it drives lexing/preprocessing/parsing/name-resolution/type-checking through the database and returns, for each Verilog-A module, a fully type-checked description — a `Vec<ModuleInfo>`. If any diagnostic is fatal it returns `None` and compilation stops. This is Chapters 4–6.
- **osdi::compile(...)** runs *everything else*: for each `ModuleInfo` it builds the MIR, discovers the DAE topology, differentiates, optimises, generates LLVM IR, and writes both the machine code and the OSDI descriptor. This is Chapters 7–13. (The heavy per-module work happens in `CompiledModule::new`, called from inside `osdi::compile`; the DAE builder it invokes still lives in the `sim_back` crate — see Chapter 9.)

So the front-end/back-end boundary is “*type-checked module description*” (`ModuleInfo`): before it, everything is about understanding the source; after it, everything is about turning one module’s meaning into numeric code.

The `--dump-mir`, `--dump-unopt-mir`, `--dump-ir`, `--dump-unopt-ir` flags you can pass on the com-

mand line are threaded straight through so you can print each IR at each stage — we use them for the worked examples later.

2.3 Why so many intermediate representations?

Beginners often ask why a compiler doesn't just translate source straight to machine code. The answer is separation of concerns: each IR is shaped for the questions asked at that stage.

IR	Shaped for	Knows about
CST / AST	faithfully representing <i>what the programmer wrote</i> (including whitespace & errors)	Verilog-A syntax
HIR	<i>meaning</i> : name resolution, types, disciplines/natures	Verilog-A semantics
MIR	<i>analysis & transformation</i> : differentiation, optimisation	pure numeric dataflow (SSA)
LLVM IR	<i>machine code generation</i>	registers, calling conventions, target CPU

Each lowering *throws away* information the next stage doesn't need. By the time we reach MIR, the code no longer knows the difference between a user variable and a temporary — it is just a graph of arithmetic. That is precisely what makes automatic differentiation and optimisation tractable.

Chapter 3 — Background you'll need

This chapter front-loads the four ideas the rest of the document reuses constantly. If you already know DAEs, Newton's method, SSA form, and incremental compilation, skim it. Otherwise read carefully — every later chapter assumes these.

3.1 The device as a system of equations: $\mathbf{I}(\mathbf{x}) + d/dt \mathbf{Q}(\mathbf{x}) = \mathbf{0}$

From §1.2, the simulator builds one equation per node from Kirchhoff's Current Law. OpenVAF states the *whole module's* contribution to that system in one compact form, written in the source as a doc-comment on `DaeSystem` (`openvaf/sim_back/src/dae.rs`):

$$\mathbf{I}(\mathbf{x}) + d/dt \mathbf{Q}(\mathbf{x}) = \mathbf{0}$$

Read it like this:

- $\mathbf{x}$ is the vector of unknowns — the quantities the simulator is solving for. For a plain device these are the node voltages at its terminals (plus a few internal nodes the model may introduce). In OpenVAF each unknown is a `SimUnknown`.
- $\mathbf{I}(\mathbf{x})$ is the resistive residual: the instantaneous current each equation demands as a function of the unknowns. Ohm's law lives here: $\mathbf{I} = \mathbf{V}/\mathbf{R}$.
- $\mathbf{Q}(\mathbf{x})$ is the reactive residual (charges/fluxes) whose time derivative is a current. A capacitor's $Q = C \cdot V$ lives here; the simulator differentiates it in time. In Verilog-A you write reactive behaviour with `ddt(...)`, e.g. `I(a,b) <+ ddt(C*V(a,b))`.
- The equation $\mathbf{I} + d\mathbf{Q}/dt = \mathbf{0}$ is just KCL for the whole system: the resistive currents plus the rate of change of the stored charges must sum to zero at every node.

To solve this non-linear system the simulator uses Newton's method. The doc comment spells out the iteration:

$$\mathbf{J}_{\mathbf{I}}(\mathbf{x}) \cdot \Delta \mathbf{x} = \mathbf{I}(\mathbf{x}) + d/dt \mathbf{Q}(\mathbf{x})$$

$$\mathbf{x}' = \mathbf{x} - \Delta \mathbf{x}$$

where $\mathbf{J}_{\mathbf{I}}(\mathbf{x})$ is the Jacobian — the matrix of partial derivatives $\partial(\text{residual}_i)/\partial(\text{unknown}_j)$. Each Newton step solves a *linear* system for the correction $\Delta \mathbf{x}$, then updates the guess. It repeats until $\mathbf{I} + d\mathbf{Q}/dt$ is essentially zero.

This is why OpenVAF exists. A device model, compiled to OSDI, must be able to hand the simulator, on demand and for the current guess $\mathbf{x}$:

1. the residual vector pieces $\mathbf{I}(\mathbf{x})$ and $\mathbf{Q}(\mathbf{x})$ (the `<+>` contributions), and
2. the Jacobian pieces $\partial \mathbf{I} / \partial \mathbf{x}$ and $\partial \mathbf{Q} / \partial \mathbf{x}$.

Item 2 is produced by automatic differentiation (Chapter 6). The whole DAE object OpenVAF builds for a module is:

```
// openvaf/sim_back/src/dae.rs (trimmed)
pub struct DaeSystem {
    /// the unknowns x being solved for
    pub unknowns: TiSet<SimUnknown, SimUnknownKind>,
    /// per-unknown residual: the (resistive I, reactive Q) contributions
    pub residual: TiVec<SimUnknown, Residual>,
```

```

    /// the Jacobian entries  $J_{ij} = (dI_i/dx_j, dQ_i/dx_j)$ 
    pub jacobian: TiVec<MatrixEntryId, MatrixEntry>,
    pub noise_sources: Vec<NoiseSource>,           // for .noise analysis
    pub num_resistive: u32,
    pub num_reactive: u32,
    // ...
}

```

and each unknown is one of just three kinds (openvaf/sim_back/src/lib.rs):

```

pub enum SimUnknownKind {
    KirchhoffLaw(Node),           // a node voltage (a KCL equation at that node)
    Current(CurrentKind),         // a branch current treated as an extra unknown
    Implicit(ImplicitEquation),   // an extra equation the model introduced (e.g. idt)
}

```

Keep $I(x) + dQ/dt = 0$ in your head; the middle third of this document is really just “how do we build, differentiate, and emit those two functions and their derivatives”.

3.2 SSA form (Static Single Assignment) — the shape of the MIR

Once the analog block is a pure numeric function, OpenVAF represents it in SSA form, the same idea LLVM and Cranelift use. Two rules define SSA:

1. Every variable is assigned exactly once. If the source reassigns x , SSA invents a fresh name ($x_1, x_2, \dots$) for each assignment.
2. Where control flow merges (after an `if`, or at a loop head), a special φ (“phi”) node picks the right incoming value depending on which path was taken.

Example. This Verilog-A:

```

x = a * b;
if (c > 0)
    x = x + 1;
y = x * x;

```

becomes, conceptually, this SSA (real MIR is shown in Chapter 5):

```

    x1 = fmul a, b
    br (c > 0) -> then, else
then:
    x2 = fadd x1, 1.0
    jmp merge
else:
    jmp merge
merge:
    x3 = phi [x2 from then], [x1 from else]    ; pick whichever ran
    y1 = fmul x3, x3

```

Why bother? Because SSA makes dataflow explicit: each value has exactly one definition, so “what does this value depend on?” is answered by following its single definition backward. That property is what makes automatic differentiation and optimisation (constant folding, common-subexpression elimination, dead-code elimination) simple and fast. The MIR crate’s own module doc says it plainly: SSA “allows very efficient implementations of the various algorithms in the backend.”

The MIR is also deliberately untyped and decoupled from the AST: an instruction like `fadd` always takes two floats and returns a float; there is no lingering notion of “user variable named x ”. This is what makes the back end fast and language-agnostic.

3.3 The Jacobian and automatic differentiation, intuitively

The simulator needs $\partial(\text{each current})/\partial(\text{each voltage})$. There are three ways a compiler could get these:

- Finite differences — nudge a voltage by ε , recompute, divide. Cheap to implement, but *inaccurate* (cancellation error) and *slow* (one extra full evaluation per voltage). Real simulators avoid this for compiled models.
- Symbolic differentiation of the source — differentiate the mathematical expressions. Exact, but naïvely explodes in size (“expression swell”).
- Automatic differentiation (AD) on the SSA graph — apply the chain rule mechanically to each instruction, reusing shared subexpressions. Exact *and* compact.

OpenVAF uses AD on the MIR (`openvaf/mir_autodiff`). Because the MIR is SSA, AD is a graph walk: for each instruction it knows the local derivative rule ($d(a*b) = a \cdot db + b \cdot da$, $d(\exp x) = \exp(x) \cdot dx$, ...) and it stitches those into new MIR instructions that compute the derivative *alongside* the original value, sharing everything that can be shared. The result is that $\partial I / \partial V$ is itself just more MIR — which then gets optimised and compiled exactly like the residual. Chapter 6 is the details.

3.4 Incremental compilation with **salsa** — the “database”

OpenVAF is structured as a set of queries over a database, using the **salsa** framework (the same idea powers `rust-analyzer`). Instead of a straight-line `main()` that does step after step, the compiler defines memoised functions like “give me the parse tree of file *F*”, “give me the type of expression *E*”, “give me the item tree of module *M*”. The first time you ask, it computes and caches the answer; ask again and you get the cached value.

Two database layers stack up (see `CLAUDE.md` and Chapter 15):

- **BaseDB** (`openvaf/basedb`) — the *syntactic* layer: the virtual file system, file texts, preprocessing, lexing, parsing, and linting.
- **CompilationDB** (`openvaf/hir`) — extends BaseDB with the *semantic* layer: HIR items, name resolution, type inference.

For a batch compiler that runs once and exits, incrementality mostly buys clean laziness: nothing is computed until something downstream asks for it, and shared sub-results (a parsed header, a resolved discipline) are computed once and reused across every module that needs them. When you saw `CompilationDB::new_fs(...)` in `compile()`, that call *set up* the database; the actual lexing/parsing happens lazily the first time `collect_modules` queries it.

With these four ideas — the DAE system, SSA, automatic differentiation, and the query database — you have the scaffolding. Now we walk the pipeline.

Chapter 4 — Stage 1: reading and preprocessing the source

Crates: `openvaf/vfs`, `openvaf/preprocessor`, `openvaf/lexer`, `openvaf/tokens`.

4.1 The virtual file system (**vfs**)

The compiler never touches the real filesystem directly during analysis. Instead every source file is loaded into a virtual file system and assigned a small integer id, a `FileId`. All later stages refer to files by `FileId`, and every piece of source is identified by a *span* — a `(FileId, text range)` pair. This indirection is what lets the compiler point diagnostics at exact source locations, and it is what makes the incremental database work (a query keyed on `FileId` can be re-run when only that file changes).

4.2 Lexing: text $\rightarrow$ raw tokens (**lexer**, **tokens**)

Lexing (a.k.a. tokenising) chops the raw character stream into atomic tokens: identifiers, numbers, keywords, punctuation, comments, whitespace. The low-level scanner is `openvaf/lexer` (a hand-written cursor over the bytes). Its output is classified into the compiler's `SyntaxKind` by `openvaf/tokens`. For example (`openvaf/tokens/src/lib.rs`):

```
// raw lexer token kind -> parser-level SyntaxKind
LineComment | BlockComment { terminated: true } => SyntaxKind::COMMENT,
Whitespace                                     => SyntaxKind::WHITESPACE,
SimpleIdent => SyntaxKind::from_keyword(src).unwrap_or(SyntaxKind::IDENT),
EscapedIdent => SyntaxKind::IDENT,
```

Two things worth noticing for a beginner:

- Keywords are just identifiers that matched a table. `from_keyword("module")` returns `MODULE_KW`; `from_keyword("my_var")` returns `None`, so it stays a plain `IDENT`. This is why an identifier accidentally named like a keyword can surprise you.
- Comments and whitespace are kept as tokens, not discarded. That is a hallmark of a *lossless* front end (see §5.1): the token stream can reproduce the original text byte-for-byte.

4.3 Preprocessing: expanding macros and includes (**preprocessor**)

Verilog-A has a C-like preprocessor. Before real parsing, the preprocessor crate handles the backtick directives:

- ``include "disciplines.vams"` — splice another file's tokens in at this point.
- ``define GMIN 1e-12` and ``define VT(T) ((`P_K)*(T)/(`P_Q))` — object-like and function-like macros, expanded on use.
- ``ifdef / `ifndef / `else / `endif` — conditional compilation.

The output is a `Preprocess` value (`openvaf/preprocessor/src/lib.rs`):

```
pub struct Preprocess {
    pub ts:          Arc<Vec<Token>>, // the fully-expanded token stream
    pub sm:          Arc<SourceMap>,  // maps each token back to its *original*
    ↪ file+range
    pub diagnostics: Arc<Diagnostics>, // any errors found while preprocessing
}
```

The source map (sm) is the important subtlety. After a macro expands, the resulting tokens did not literally appear at that spot in the file — but for good error messages the compiler must still be able to say “this came from line 12 of `disciplines.vams`, included at line 1 of your model”. The `SourceMap` records that provenance for every token so a diagnostic can be traced through include and macro layers back to something the user actually typed.

Because the preprocessor is text-level and runs before parsing, a few Verilog-A features are implemented here as *textual pre-passes* rather than in the grammar — for instance `__FILE__`/`__LINE__` substitution, and some legacy syntaxes normalised into their modern spelling before the parser ever sees them.

Robustness matters. Preprocessors are a classic source of infinite loops — a macro that expands to itself, or a `//` comment at end-of-file with no trailing newline. The reloaded compiler guards these (bounded macro-recursion depth; an explicit end-of-input arm in the comment scanner) so that malformed input produces a *diagnostic*, not a hang.

At the end of Stage 1 we have a flat, fully-expanded stream of **SyntaxKind** tokens, each still traceable to its origin. That stream feeds the parser.

Chapter 5 — Stage 2: parsing into a syntax tree

Crates: `openvaf/parser`, `openvaf/syntax` (built on the `rowan` library).

5.1 Concrete vs. abstract syntax trees

Parsing turns the flat token stream into a tree that mirrors the grammar: a module contains ports and items; an item may be an analog block; the block contains statements; a statement contains expressions; and so on.

OpenVAF (like `rust-analyzer`) builds a lossless Concrete Syntax Tree (CST) using the `rowan` library. "Lossless" means every byte of the original source — including whitespace, comments, and even syntax errors — is represented in the tree. You can walk the CST and print it back out to get the exact input. This is invaluable for tooling and for precise error messages, and it means parsing does not fail catastrophically on a mistake: it inserts an error node and keeps going, so the compiler can report *several* problems in one run instead of stopping at the first.

On top of the untyped CST sits a typed AST layer (`openvaf/syntax/src/ast/`). The CST is a uniform tree of nodes tagged with a `SyntaxKind`; the AST wraps particular node kinds in Rust types with convenient accessors. For example the generated AST has:

```
// openvaf/syntax/src/ast/generated/nodes.rs (excerpt)
pub struct ModuleDecl { /* wraps a MODULE_DECL syntax node */ }
pub struct ModulePorts { ... }
pub struct ModulePort { ... }
pub struct BranchDecl { ... }
```

A `ModuleDecl` is just a *view* onto a CST node whose kind is `MODULE_DECL`; asking it for `.ports()` navigates to the child `MODULE_PORTS` node. Nothing is copied — the AST is a thin, typed lens over the CST.

5.2 The parser itself

The entry point is `tiny` (`openvaf/parser/src/lib.rs`):

```
pub fn parse(tokens: &[SyntaxKind]) -> Output {
    let mut p = Parser::new(tokens);
    grammar::source_file(&mut p); // the whole grammar starts here
    p.finish()
}
```

`grammar::source_file` is a hand-written recursive-descent parser: one Rust function per grammar rule (`module`, `port_declaration`, `analog_stmt`, `expr`, ...), each consuming tokens and emitting tree-building events (`start_node`, `token`, `finish_node`). Recursive descent is chosen because it gives precise control over error recovery and produces good diagnostics.

Operator precedence (so that `a + b * c` parses as `a + (b*c)`) is handled with a Pratt parser inside the expression grammar — a small precedence table assigns each operator a binding power. Getting that table to match the Verilog-A LRM exactly is fiddly (e.g. `%` binds like `*` and `/`; unary operators bind tighter than `**`, so `-2**2` is 4, not -4), and it is one of the places the reloaded fork has been carefully audited.

5.3 What comes out

The product of Stage 2 is a CST for the whole file, with a typed AST overlay, still fully connected to source spans through the source map. Crucially, the tree is *purely syntactic*: it knows that **V(p,n)** is a call-like expression with two arguments, but it does not yet know that **V** means “the potential of a branch”, that **p** and **n** are electrical nodes, or that the expression’s type is **real**. Establishing all of that meaning is the job of the HIR.

Chapter 6 — Stage 3: the HIR (meaning, names, and types)

Crates: `openvaf/hir_def` (definitions, name resolution), `openvaf/hir_ty` (type inference & checking), `openvaf/hir` (the CompilationDB facade).

The HIR (“High-level Intermediate Representation”) is where the compiler stops thinking about *syntax* and starts thinking about *meaning*. Three questions get answered here: *what items exist?*, *what does each name refer to?*, and *what is the type of every expression?*

6.1 The item tree: what’s declared (`hir_def::item_tree`)

The first HIR step condenses the sprawling CST into a compact item tree — a summary of the *declarations* in a file, independent of the details inside function bodies. It records the modules, their ports and nets, branches, parameters, variables, functions, disciplines, and natures. Splitting “declarations” from “bodies” is a deliberate performance move borrowed from `rust-analyzer`: editing the inside of one analog block shouldn’t force re-analysis of every other module’s *signature*.

For a device, the item tree captures things like: *module resistor has ports p, n; both are electrical; there is a parameter R : real = 1000*. Port widths and parameter-dependent array sizes are also resolved structurally here (this is where a declaration like `real x[0:N-1]` gets its length pinned down from N’s default).

6.2 Name resolution: what each identifier means (`hir_def::nameres`)

Name resolution (“nameres”) maps every identifier to the thing it denotes. In $I(p, n) \leftarrow V(p, n)/R$:

- **p** and **n** resolve to the module’s two electrical nodes;
- **V** resolves to the potential access function of the `electrical` discipline, and **I** to its flow access function;
- **R** resolves to the parameter;
- **(p, n)** resolves to the branch between those two nodes.

Resolution respects scopes (module scope, block scope, function scope) and the imported disciplines/natures (from ``include "disciplines.vams"`). It is also where the compiler knows about disciplines and natures — the Verilog-A abstraction that says “electrical potential is measured in volts with this tolerance, electrical flow is current in amps”, so that `V(...)` and `I(...)` mean the right physical quantities. Undeclared identifiers used as instance connections become *implicit nets*, and genuine unknown names become diagnostics here.

6.3 Built-in functions (`hir_def::builtin`)

Verilog-A’s system and math functions — `ddt`, `idt`, `$temperature`, `sqrt`, `exp`, `white_noise`, `$strobe`, `$stable_model`, and so on — are enumerated in one big registry:

```
// openvaf/hir_def/src/builtin.rs (excerpt)
pub enum BuiltIn {
    // ...
    absdelay, ddt, idt, idtmod, ...
    sqrt, exp, ln, log, ...
    // hundreds more
}
```

Each built-in is registered so that name resolution can recognise a call to it, and so the type checker (next) can look up its signature. Adding a new Verilog-A built-in is, mechanically, “add a variant here, give it a signature in `hir_ty`, and lower it in `hir_lower`” — a recipe used repeatedly to close gaps against the LRM.

6.4 Type inference and checking (`hir_ty`)

Verilog-A has a small type system — essentially real, integer, string, arrays of those, plus language-level categories like “a node”, “a branch”, “a nature”, “a parameter”. `hir_ty` walks each expression bottom-up and assigns it a type, checking that operators and function arguments get what they expect.

Two enums capture the vocabulary. `Ty` is the type an expression *has*; `TyRequirement` is what a context *demands* (`openvaf/hir_ty/src/types.rs`):

```
pub enum TyRequirement {
    Val(Type),           // a value of a specific type (real/integer/string)
    Condition,           // something usable as an if-condition
    AnyVal,
    ArrayAnyLength { ty: Type },
    Node,                // must be a node (e.g. the argument of V(...))
    PortFlow,
    Nature,
    Var(Type), Param(Type), AnyParam,
    Branch,              // must be a branch
    Literal(Type),
    Function,
}
```

This is how the checker enforces that `V(p, n)`’s arguments must be nodes, that the `if` in `if (c > 0)` gets a condition, and that `R` used in a division is a numeric value. Type mismatches (a string where a real is required, a branch used as a number, calling a function with the wrong arity) turn into user-facing diagnostics. Implicit conversions the LRM allows — e.g. an integer used where a real is expected — are inserted here as explicit casts so later stages never have to guess.

The checker also does *language-rule validation* that isn’t strictly about types: you can’t contribute to a branch whose nodes are all ground; an analog event control must be well-formed; a discipline’s domain must be consistent. These live in `hir_ty/src/validation/`.

6.5 What the HIR gives the back end

After Stage 3 the compiler has, for each module, a fully resolved and type-checked semantic model: every name bound, every expression typed, every branch and node and parameter identified, and every built-in call recognised. Diagnostics accumulated so far are reported; if any is fatal, `collect_modules` returns `None` and `compile()` stops (that is the `FatalDiagnostic` path you saw in Chapter 2).

What the HIR is *not* is something you can differentiate or optimise. It is still tree-shaped, still tied to language concepts, still carrying control flow as nested statements. To do numerical work we lower it into the MIR — and, at the same time, discover the device's DAE topology. That double act is the subject of the next two chapters.

Chapter 7 — Stage 4: lowering HIR to MIR

Crate: `openvaf/hir_lower` (with `openvaf/mir_build` doing the SSA construction underneath). This crate's own doc calls it *"the only bridge between the various MIR crates and the HIR"* — everything language-specific stops here; everything numeric starts here.

7.1 The core idea: turn the analog block into a pure function

Lowering translates the body of `analog begin ... end` into an SSA Function (Chapter 8). Sequential statements become instructions; `if/case/loops` become basic blocks and branches; expressions become arithmetic instructions. A local Verilog-A variable is *not* a memory cell in the MIR — thanks to SSA construction, each assignment produces a fresh value, and phi nodes reconcile values at control-flow merges (`mir_build` implements the standard SSA-construction algorithm so the rest of the compiler never sees mutable variables).

But a device model is not a self-contained program: it reads quantities the simulator provides (voltages, temperature, time) and produces quantities the simulator consumes (branch contributions, operating-point variables). Lowering therefore has to record, precisely, *which SSA values mean which simulator-level things*. That bookkeeping is the `HirInterpreter`.

7.2 The **HirInterpreter**: the input/output contract

This one struct (`openvaf/hir_lower/src/lib.rs`) is the most important object to understand in the whole front end:

```
pub struct HirInterpreter {
    /// OUTPUTS: which SSA Value computes each "place" the simulator reads back
    /// (a branch contribution, a variable, a residual, bound_step, ...).
    pub outputs: IndexMap<PlaceKind, PackedOption<Value>, ...>,

    /// INPUTS: which SSA Param stands for each simulator-provided quantity
    /// (a node voltage, a branch current, temperature, $abstime, ...).
    pub params: TiMap<Param, ParamKind, Value>,

    /// external functions the generated code calls back into the simulator for
    /// (noise sources, $strobe/$display, file I/O, $limit, ...).
    pub callbacks: TiSet<FuncRef, CallbackKind>,

    pub tagged_reads: IndexMap<Value, Variable, ...>,
    // ...
}
```

Think of the lowered function as `eval(inputs) -> outputs`, where **params** names the inputs and **outputs** names the outputs. Everything the simulator will later feed in or read out is enumerated in these two maps.

Inputs — **ParamKind**. An MIR function parameter is not "argument #3"; it is a *meaning*. The `ParamKind` enum (`openvaf/hir_lower/src/lib.rs`) lists every kind of quantity the simulator supplies:

```

pub enum ParamKind {
    Param(Parameter), // a model/instance parameter, e.g. R
    Voltage { hi: Node, lo: Option<Node> }, // V(hi,lo): a branch/node voltage
    Current(CurrentKind), // I(branch): a branch current unknown
    Temperature, // $temperature
    Abstime, // $abstime (transient time)
    ParamGiven { param: Parameter }, // was this parameter set by the netlist?
    PortConnected { port: Node }, // is this optional port wired up?
    ParamSysFun(ParamSysFun), // $vt, $mfactor, simulator params, ...
    HiddenState(Variable), // a variable that persists across eval() calls
    ImplicitUnknown(ImplicitEquation), // an extra unknown the model introduced
    EnableIntegration, EnableLim,
    PrevState(LimitState), NewState(LimitState), // for $limit / device limiting
    IsInitialStep, IsFinalStep, // @(initial_step) / @(final_step) gates
    EventState(u32), // persistent edge-detector state (cross/timer)
    // ...
}

```

So when the lowered code needs “the voltage across branch (p,n)”, it does not compute it — it *requests* it as a parameter of kind `Voltage { hi: p, lo: Some(n) }`. The interner hands back the SSA Value representing that input, creating it on first request. This is the mechanism by which `V(p,n)` in the source becomes a leaf of the dataflow graph.

Outputs — **PlaceKind**. Symmetrically, every result the simulator can read back is a `PlaceKind`:

```

pub enum PlaceKind {
    Var(Variable), // an operating-point / output variable
    FunctionReturn(hir::Function), // a user analog-function's return slot
    Contribute { dst: BranchWrite, reactive: bool, voltage_src: bool },
    // an I(...)<+ or V(...)<+ contribution
    ImplicitResidual { equation: ImplicitEquation, reactive: bool },
    Param(Parameter), ParamMin(Parameter), ParamMax(Parameter), // during param
    ↪ init
    BoundStep, // $bound_step time-step limit
    EventState(u32), // write side of an edge detector
    // ...
}

```

The crucial one is **Contribute**. The statement `I(p,n) <+ expr` records, in outputs, that the *resistive* contribution to branch (p,n) is the SSA value computing `expr`. If the source instead wrote `I(p,n) <+ ddt(expr)`, the `ddt` marks it reactive, and it is recorded with `reactive: true` — that is exactly how the `I` (resistive) and `Q` (reactive) halves of the DAE from Chapter 3 get separated. `voltage_src: true` marks the case `V(p,n) <+ ...`, where the model imposes a voltage instead of a current (a different kind of equation).

7.3 Callbacks: talking back to the simulator

Some Verilog-A features can't be pure arithmetic — they have side effects or need runtime services: `white_noise(...)`, `$strobe/$display`, `$fopen/$fscanf`, `$limit`, `analysis("ac")`. These lower to calls to callbacks, enumerated in `CallbackKind` and interned in `callbacks`. In the MIR they appear as `Call` instructions to opaque function references; at codegen time they become calls into simulator-provided (or OpenVAF runtime `stdlib`) functions. Keeping them as explicit callbacks means the optimiser and differentiator can treat them as opaque (a callback with no derivative simply contributes zero to the Jacobian).

7.4 The result of lowering

Lowering a module produces (roughly): one MIR **Function** for the evaluation (the analog block), companion functions for parameter initialisation and model/instance setup, and the fully populated **HirInterner** that says what every input and output means. At this point the code is pure SSA dataflow with clearly labelled boundary values — ready both for the *topology analysis* that builds the DAE (Chapter 9) and for the *differentiation* that builds the Jacobian (Chapter 10). First, though, let’s look at what the MIR actually is.

Chapter 8 — The MIR in detail (SSA, dataflow, opcodes)

Crate: `openvaf/mir` (data structure), `openvaf/mir_build` (construction), `openvaf/mir_reader/write` (text form for tests/debugging).

The MIR is OpenVAF’s own SSA intermediate representation, “heavily inspired by the IR in Cranelift and LLVM” but simplified for the analog domain: it only needs to express Verilog-A arithmetic, so it drops everything about hardware opcodes, atomics, memory models, and types (§8.4).

8.1 A **Function** is four tables

```
// openvaf/mir/src/lib.rs (trimmed)
pub struct Function {
    pub name: String,
    pub dfg: DataFlowGraph, // all values, instructions, blocks, and their data
    pub layout: Layout,     // the ORDER of blocks, and of instructions within
    ↪ them
    // + source-location side table
}
```

The separation of **dfg** (what the instructions *are* and how they connect) from **layout** (what *order* they run in) is a Cranelift-ism: it lets passes reorder or splice instructions cheaply without touching their definitions. The main entities (`openvaf/mir/src/entities.rs`) are:

- **Value** — an SSA value (the result of one instruction, or a function parameter, or a constant). Written `v0`, `v1`,
- **Inst** — an instruction. Written `inst5`, but usually shown by its result: `v7 = fadd v3, v6`.
- **Block** — a basic block: a straight-line run of instructions ending in a branch/jump/return. Written `block0`, `block1`,
- **Param** — a formal parameter value (an input; see `ParamKind` in Chapter 7).
- **FuncRef** — a reference to a callable (a callback or a helper function).

8.2 The data-flow graph and **InstructionData**

The `DataFlowGraph` (`dfg`) stores, for every instruction, an `InstructionData` describing its opcode and operands, plus which `Value(s)` it defines. Most instructions are simple “opcode + a few value operands”; a handful carry extra data:

```
// conceptually, the shapes an instruction can take:
Unary   { opcode, arg }           // e.g. Fneg, Sqrt, Exp
Binary { opcode, args: [a, b] }  // e.g. Fadd, Fmul, Ilt
Branch { cond, then_block, else_block }
Jump   { destination }
```

```
Call    { func_ref, args: ValueList } // a callback / helper call
PhiNode { ... }                      // SSA merge
```

Because everything is SSA, a Value has exactly one definition; `dfg.value_def(v)` tells you whether `v` came from an instruction, a parameter, or a constant. Walking definitions backward is how autodiff and optimisation traverse the program.

8.3 The opcode set

The MIR opcode enum (`openvaf/mir/src/instructions/generated.rs`) is small and fixed — every analog computation is expressed with these. Grouped by purpose:

Group	Opcodes
Unary / casts	Inot Bnot Fneg Ineg · FICast IFcast BICast IBcast FBcast BFCast (float/int/bool conversions) · OptBarrier
Real math (unary)	Sqrt Exp Ln Log Clog2 Floor Ceil Sin Cos Tan Asin Acos Atan Sinh Cosh Tanh Asinh Acosh Atanh
Integer arithmetic	Iadd Isub Imul Idiv Irem · Ishl Ishr Iashr (shifts) · Ixor Iand Ior (bitwise)
Real arithmetic	Fadd Fsub Fmul Fdiv Frem
Comparisons	integer Ilt Igt Ige Ile; real Flt Fgt Fge Fle; equality Ieq Feq Seq(string) Beq(bool); inequality Ine Fne Sne Bne
Real math (binary)	Hypot Atan2 Pow
Control flow	Br (conditional) Jmp (unconditional) Exit (return) Call Phi

The I/F/B/S prefixes mean Integer, Float(real), Bool, String. So `Fmul` is real multiply, `Iadd` is integer add, `Feq` is real equality. This tidy, prefixed naming is a direct consequence of the MIR being untyped (next).

8.4 “Untyped” MIR — what that means and why it matters

The MIR module doc states: *“The MIR is untyped. All opcodes have fixed argument/return types.”* There is no type field on a Value; instead the opcode determines the types. `Fadd` *always* takes two reals and returns a real; `Iadd` *always* takes two integers. Type checking already happened in `hir_ty` (Chapter 6), and any needed conversions were made explicit as cast opcodes (`IFcast`, `FICast`, ...) during lowering. So by the time we reach MIR, “is this legal?” is settled and the back end can focus purely on *transforming* the dataflow. This is a big reason the back end is fast.

A subtle consequence you will meet if you ever hack on the compiler: because opcodes are strongly typed by construction, an integer-typed persistent state slot must be lowered with integer opcodes and stored in an integer-typed cell — mixing an integer value into a float slot is a bug the untyped IR won’t catch for you.

8.5 Constants and the “OptBarrier”

Common constants are interned (`F_ZERO`, `F_ONE`, ...) so the optimiser can recognise them by identity — you saw `F_ZERO` used in the DAE sparsification code in Chapter 3 (`factor == F_ZERO` ⇒ drop this contribution). The `OptBarrier` opcode is a deliberate *fence*: it wraps a value to stop certain optimisations from moving or merging across it, which the compiler uses to protect values that must survive until a later stage reads them out (for example, an output value the OSDI layer will attach to a specific descriptor slot). `strip_optbarrier(...)` peels the fence back off when the real value is finally needed.

8.6 Reading MIR

You can print the MIR of any model with the CLI flags from Chapter 2:

```
openvaf-r --dump-unopt-mir resistor.va  # right after lowering, before  
↪ optimisation  
openvaf-r --dump-mir      resistor.va  # after optimisation + autodiff
```


The textual form (produced by `openvaf/mir/src/write.rs`) looks like the SSA sketch in §3.2: **blockN**: labels, one **vN = opcode args** per line, ending each block with **br/jmp/exit**. We use this in the worked example (Chapter 14) to see a real device's residual and its machine-generated derivatives side by side.

Chapter 9 — Stage 4 (cont.): building the DAE system (`sim_back`)

Crate: `openvaf/sim_back`. Both the front-end entry `collect_modules()` and the DAE builder live here. The lowered SSA plus the interner become an actual device: a set of DAE equations (`DaeSystem` from Chapter 3) inside `CompiledModule::new` — which the back end (`osdi::compile`) calls once per module. So although this analysis conceptually belongs to "Stage 4", it is physically driven from the back end; the boundary in `compile()` (Chapter 2) is deliberately just the type-checked `ModuleInfo`.

9.1 From contributions to topology (`sim_back/src/topology.rs`)

Lowering gave us a bag of `Contribute` outputs: "resistive contribution to branch (p,n) is value v42", "reactive contribution to branch (a,b) is value v88", and so on. Turning that bag into a coherent circuit topology is surprisingly involved, because Verilog-A is implicit in places. The `topology` module (quoting its own header) does three main jobs:

1. Linearise or split **ddt** and noise. A `ddt(x)` either becomes a *reactive contribution* directly (the common case — its argument is the charge Q), or, if it appears somewhere that can't be a simple reactive term, it becomes an implicit internal equation with its own unknown. `white_noise/flicker_noise/noise_table` become noise sources attached to the branch. `idt` (time integral) becomes an implicit equation as well.
2. Find the small-signal network. Some nodes are *statically* always zero volts in a large-signal (DC/transient) analysis — they only carry AC/noise stimulus. Detecting these lets the compiler avoid generating pointless derivatives for them.
3. Prune small-signal from large-signal contributions so that an AC-only stimulus doesn't drag unnecessary derivative computation into the DC path.

The output is a clean `Topology`: for each branch, its resistive/reactive contributions and noise; plus a list of implicit equations. That feeds the DAE builder.

9.2 The DAE builder (`sim_back/src/dae.rs`)

`DaeSystem::new` consumes the topology and produces the `DaeSystem` (Chapter 3): the set of unknowns, one residual per unknown (its resistive I and reactive Q values), the Jacobian entries (which row/column pairs are non-zero, and the values $\partial I / \partial x$, $\partial Q / \partial x$), and the noise sources. Each unknown is a `SimUnknownKind`:

- **KirchoffLaw(node)** — the ordinary case: one KCL equation per electrical node. Its residual is the sum of branch currents flowing into that node.
- **Current(branch)** — when a branch current must itself be an unknown (e.g. a voltage source `V(a,b) <+ ...`, an ideal ammeter, or a port current `I(<p>)`), the current is promoted to a solved variable with its own equation.
- **Implicit(equation)** — an extra internal equation the model created, e.g. from `idt`, `idtmod`, `absdelay`, or an implicit `ddt`.

Two matching enums describe the *reactive* and *noise* pieces (`sim_back/src/noise.rs`):

```
pub enum NoiseSourceKind {
    WhiteNoise { pwr: Value },           // white_noise(pwr)
```

```

FlickerNoise{ pwr: Value, exp: Value },           // flicker_noise(pwr, exp)
NoiseTable { log: bool, vals: Box<[(Ieee64, Ieee64)]> }, // tabulated S(f)
AcStim      { mag: Value, phase: Value },          // ac_stim small-signal source
}

```

9.3 Node collapse (`sim_back/src/node_collapse.rs`)

Real models frequently contain nodes that *sometimes* vanish. A classic pattern: a series resistance R_s between an external terminal and an internal node, where if the model card sets $R_s = 0$ the internal node should merge with the external one — otherwise the matrix has a redundant unknown and a zero-pivot. Verilog-A expresses this by *conditionally* contributing (or by the $V(a,b) <+ 0$ idiom), and OpenVAF detects it as node collapse: a pair of `SimUnknown`s that, under the right parameter values, become the same unknown.

Collapse is data (`NodeCollapse`: a set of (`SimUnknown`, `Option<SimUnknown>`) pairs), evaluated per instance at setup time, so the simulator can build a *smaller* matrix when a node collapses. Getting collapse right is delicate: a branch that is *probed* (its voltage read) but *never contributed to* must read as an open circuit, while one that is contributed to and then collapsed must merge — several DAE-correctness fixes in the reloaded fork live here.

9.4 Initialisation vs. evaluation

`CompiledModule::new(sim_back/src/lib.rs)` — the per-module build the back end runs — bundles more than just the DAE:

```

pub struct CompiledModule<'a> {
    pub dae_system:      DaeSystem,      // the residual + Jacobian structure
    pub intern:          HirInterner,    // the eval() input/output contract
    pub init:            Initialization, // parameter init + setup code
    pub model_param_intern: HirInterner, // model-level parameter processing
    // ...
}

```

The split matters because a simulator calls a device at different times for different reasons:

- Parameter initialisation / setup — once per model and once per instance, when the netlist is read: apply defaults for parameters the netlist didn't set, run range checks, compute anything that depends only on parameters (temperature scaling, geometry). This is `Initialization/model_param_intern`.
- Evaluation (**eval**) — millions of times, inside the Newton loop: given the present voltages, compute residual + Jacobian. This is `intern + dae_system`.

Separating them means expensive parameter-only work is hoisted out of the hot `eval` path. The two internally use the *same* MIR machinery, just with different inputs designated as "known".

At the end of Stage 4 we have, per module, a complete mathematical description of the device: what it solves for, what it contributes, and how everything derives — but the derivatives themselves are still only *structural* (we know *which* Jacobian entries exist). Computing the derivative *values* is automatic differentiation, and it happens as the back end lowers each of these into concrete MIR functions.

Chapter 10 — Stage 6: automatic differentiation (the Jacobian)

Crate: `openvaf/mir_autodiff`. This is the chapter that most sets OpenVAF apart. The simulator needs $\partial(\text{residual})/\partial(\text{unknown})$ for every non-zero Jacobian entry; the model author wrote *none* of them. The compiler manufactures them by differentiating the MIR.

10.1 What “differentiate the MIR” means

Recall the residual is now an SSA graph of arithmetic — `Fmul`, `Fadd`, `Exp`, `Ln`, ... — whose leaves include the input parameters $V(p, n)$, $V(a, b)$, temperature, and so on. To get $\partial \text{residual} / \partial V(p, n)$, we apply the chain rule mechanically to that graph:

- The derivative of a leaf $V(p, n)$ with respect to $V(p, n)$ is 1; with respect to any *other* unknown it is 0.
- The derivative of each instruction follows a fixed local rule combined with the derivatives of its inputs — e.g. $d(a \cdot b) = a \cdot db + b \cdot da$, $d(\exp x) = \exp(x) \cdot dx$, $d(\sqrt{x}) = dx / (2 \sqrt{x})$.

Because the MIR is SSA, this is a graph walk that emits new MIR instructions computing the derivative *alongside* the original. The derivative of the residual is therefore, itself, just more MIR — which then gets optimised and compiled exactly like the residual. No finite differences, no accuracy loss; the Jacobian is symbolically exact.

10.2 What are we differentiating with respect to? **Unknowns**

The compiler tells the differentiator which values are the “variables” via `KnownDerivatives` (`openvaf/mir/src/lib.rs`):

```
pub struct KnownDerivatives {
    /// each Unknown is an SSA Value we can differentiate with respect to
    /// (a node voltage, a branch current, ...)
    pub unknowns: TiSet<Unknown, Value>,
    /// ddx() calls: user-requested derivatives, mapped to their unknown sets
    pub ddx_calls: AHashMap<FuncRef, (HybridBitSet<Unknown>,
        ↪ HybridBitSet<Unknown>)>,
}

pub struct Unknown(pub u32);
```

The unknowns are exactly the DAE unknowns from Chapter 9 (the voltages/currents). The `ddx_calls` handle the Verilog-A `ddx(f, V(a,b))` operator, where the *model author* explicitly asks for a derivative — same machinery, different trigger.

The public entry point is `small` (`openvaf/mir_autodiff/src/lib.rs`):

```
pub fn auto_diff(
    func: impl AsMut<Function>,
    dom_tree: &DominatorTree,
    derivatives: &KnownDerivatives,
```

```

    extra_derivatives: &[(Value, mir::Unknown)],
) -> HashMap<(Value, mir::Unknown), Value, ...> {
    let mut intern = DerivativeIntern::new(derivatives);
    let live = LiveDerivatives::build(func, &mut intern, extra_derivatives,
        ↪ dom_tree);
    build_derivatives(func, &mut intern, &live, dom_tree.cfg_postorder())
}

```

It returns a map: “the derivative of value V with respect to unknown U is value W ”. The DAE builder then reads that map to fill in `MatrixEntry.resist/.react` — the Jacobian entries.

10.3 Only compute derivatives you need: **LiveDerivatives**

A naïve differentiator would compute $\partial(\text{every value})/\partial(\text{every unknown})$ — quadratic blow-up. OpenVAF first runs a liveness analysis (`live_derivatives.rs`) that figures out which (value, unknown) derivatives can *actually reach* a Jacobian output. Only those are built. This is the single most important optimisation in the differentiator: for a transistor with dozens of unknowns, the vast majority of cross-derivatives are structurally zero (an equation for terminal A doesn’t depend on an unrelated internal node), and computing them would be pure waste. `LiveDerivatives::build` walks the graph in post-order propagating “this instruction’s derivative w.r.t. unknown U is needed”, so the builder only materialises live ones.

10.4 The local derivative rules

`build_derivatives` (in `builder.rs`) contains one rule per differentiable opcode. A representative slice (`openvaf/mir_autodiff/src/builder.rs`), where `arg0` is the operand and `res` is the original result:

```

Opcode::Exp    => self.ins().exp(arg0),           // d/dx exp(x) = exp(x)
Opcode::Ln     => arg0,                           // d/dx ln(x) = 1/x  (as
    ↪ factor 1/arg0)
Opcode::Sqrt   => self.ins().fmul(F_TW0, res),     // d/dx √x = 1/(2√x)
Opcode::Sin    => self.ins().cos(arg0),           // d/dx sin = cos
Opcode::Cos    => { /* -sin(x) */ }
Opcode::Sinh   => self.ins().cosh(arg0),
Opcode::Cosh   => self.ins().sinh(arg0),
Opcode::Fdiv   => self.ins().fmul(arg1, arg1),     // quotient-rule denominator
Opcode::Pow    => { /* x^n : n·x^(n-1) · dx (+ ln·x^n·dn if exponent varies) */ }
// ... one arm per math opcode ...

```

Each rule emits the *local* factor; the surrounding machinery multiplies it by the incoming derivative (the chain rule) and sums contributions where a value feeds multiple places. Products (`Fmul`) use $a \cdot db + b \cdot da$; sums (`Fadd`) pass derivatives straight through; a `Call` to a callback that has no registered derivative contributes zero (it’s treated as a constant with respect to the unknowns). Constants and comparisons differentiate to zero.

An important subtlety the reloaded fork got right: intrinsics used in derivatives must actually be *registered with the backend*. If a rule emits, say, `llvm.fabs` or `llvm.ceil` and that intrinsic was never declared for codegen, you get a crash at LLVM time rather than a wrong number — several fixes amount to “register the intrinsic the derivative rule needs.”

10.5 Second derivatives, limiting, and **ddx**

Some analyses need more than the first Jacobian: device limiting (bounding how fast a voltage can change between Newton iterations) needs a corrective term $\text{lim_rhs} = J(\text{lim_x}) \cdot (\text{lim_x} - x)$ —

visible in the `Residual` struct's long doc comment in Chapter 3 — which itself involves derivatives evaluated at the *limited* operating point. The differentiator is re-entrant enough to build these, and it caches interned derivatives (`DerivativeIntern`) so a value differentiated for two purposes is computed once.

10.6 Where the derivatives go

After autodiff, the module's MIR contains both the residual computations *and* all live Jacobian computations, as one big SSA graph with lots of shared subexpressions (the whole point of doing AD on SSA). That graph is redundant in places — the same product may be computed for the value and again inside a derivative — which is exactly what the optimiser cleans up next.

Chapter 11 — Stage 7: optimising the MIR

Crate: `openvaf/mir_opt`. After autodiff the MIR is correct but bloated: constants aren't folded, the same subexpression appears many times, and lots of computed values feed nothing useful. The optimiser cleans this up *before* LLVM sees it — partly for speed, but mostly because a smaller, sparser graph produces a smaller, sparser Jacobian, which the simulator then solves faster.

11.1 The classic SSA passes

`mir_opt` implements the standard textbook passes, each as a function over a `Function`:

Pass (file)	What it does	Why it helps a device model
Sparse Conditional Constant Propagation (<code>const_prop.rs</code> , <code>sparse_conditional_constant_propagation</code>)	Folds constants and prunes branches that can never be taken, in one combined pass	Parameters known at compile time (<code>if (TYPE==1)</code>) collapse to a single path
Global Value Numbering (<code>global_value_numbering/</code>)	Detects when two instructions compute the <i>same</i> value and merges them (a global form of common-subexpression elimination)	The residual and its derivatives share huge subexpressions; GVN computes each once
Instruction combining (<code>inst_combine.rs</code>)	Peephole rewrites: $x*1 \rightarrow x$, $x+0 \rightarrow x$, $x*2 \rightarrow x+x$, strength reductions, algebraic identities	Cleans up the many $\cdot 1/+0$ terms autodiff's chain rule generates
Dead-code elimination (<code>dead_code.rs</code> , <code>dead_code_aggressive.rs</code>)	Removes any instruction whose result is never used by an output	Drops derivatives that turned out unused; drops intermediate charges once linearised
CFG simplification (<code>simplify_cfg.rs</code>)	Merges straight-through blocks, deletes empty/unreachable blocks	Un-nests the control flow left over from lowering <code>if/case</code>
Const evaluation (<code>const_eval.rs</code>)	Evaluates fully-constant expressions at compile time	$\$c \log_2(16)$, $\ln(2)*T$ where T is fixed, etc.
Taint splitting (<code>split_tainted.rs</code>)	Separates values "tainted" by a special input (e.g. small-signal) from those that aren't	Keeps AC/noise-only computation out of the DC residual

Dead-code elimination is guided by an explicit set of output values — the SSA values the OSDI layer

will actually read (the residuals, Jacobian entries, opvars, bound_step). Anything that can't reach an output is deleted:

```
// openvaf/mir_opt/src/dead_code.rs
pub fn dead_code_elimination(func: &mut Function, output_values: &BitSet<Value>) {
    ↪ ... }
```

That is why the interpreter's outputs map (Chapter 7) matters so much: it *defines* what "useful" means. A contribution nobody reads, or a Jacobian entry that sparsifies to F_ZERO (Chapter 3's sparsify), is simply removed — including its row/column in the matrix.

11.2 Constant folding across the interned constants

Because common constants are interned (F_ZERO, F_ONE, F_TWO, ...), the passes recognise them by identity and apply algebraic identities aggressively. This interplay — interned constants + inst-combine + GVN — is what shrinks the naïve chain-rule output (full of $\times 1$ and $+0$) down to tight code. It is also where compile-time evaluation of the newer built-ins is pinned down: e.g. `$clog2(n)` for constant `n` must fold to the exact bit-width, and `ceil/floor/$rtoi` of constants must fold with the correct rounding, all validated against the interpreter (`mir_interpret`, used for `const_eval` and for tests).

11.3 The interpreter (`mir_interpret`)

`openvaf/mir_interpret` is a small MIR evaluator: given a function and concrete inputs, it runs the SSA graph and returns the outputs. It serves two roles: (1) *constant folding* — evaluate a fully-constant subgraph at compile time; and (2) *testing* — the reference values in many snapshot tests are produced by interpreting the MIR, so a codegen change that alters a number is caught. It must agree bit-for-bit with what the compiled code computes, which makes it a useful oracle when adding or fixing a built-in.

At the end of Stage 7 the MIR is lean: minimal instructions, minimal blocks, minimal Jacobian. Now it becomes machine code.

Chapter 12 — Stage 8: code generation with LLVM

Crate: `openvaf/mir_llvm` (bindings to LLVM 18 via `llvm-sys`), plus `openvaf/target` for machine configuration.

12.1 Why LLVM

Writing a good native code generator — register allocation, instruction selection, scheduling, target-specific tuning — is enormous. LLVM is an industrial compiler back end that does all of that. OpenVAF translates its optimised MIR into LLVM IR (LLVM's own SSA representation), then asks LLVM to optimise further and emit a native object file for the target CPU. The project pins LLVM 18.1.8.

12.2 The backend object

```
// openvaf/mir_llvm/src/lib.rs
pub struct LLVMBackend<'t> { ... }
impl LLVMBackend {
    pub fn new(codegen_opts: &[String], target: &Target,
               target_cpu: String, features: &[...]) -> Self { ... }
}
```

`LLVMBackend::new` (called from `compile()` in Chapter 2) sets up an LLVM target machine for the requested triple (e.g. `arm64-apple-darwin`, `x86_64-unknown-linux-gnu`) and optimisation level. From then on, each module's functions are translated into that machine's IR.

12.3 Translating MIR to LLVM IR

The heart is `builder.rs`, which walks each MIR Function block-by-block, instruction-by-instruction, and emits the corresponding LLVM IR:

- MIR **Values** become LLVM SSA values (LLVM is SSA too, so the mapping is direct).
- MIR **Blocks** become LLVM basic blocks; `Br/Jmp` become LLVM `br`; `Phi` becomes LLVM `phi`.
- Arithmetic opcodes map to LLVM instructions or intrinsics: `Fadd`→`fadd`, `Fmul`→`fmul`, `Exp`→`@llvm.exp.f64`, `Sqrt`→`@llvm.sqrt.f64`, `Pow`→`@llvm.pow.f64`, and so on.
- MIR **Params** (the `ParamKind` inputs) become loads from the instance/model data structs — the code reads the present voltage/temperature/parameter out of the memory block the simulator handed in.
- MIR outputs (the `PlaceKinds`) become stores into those structs, or into the residual/Jacobian arrays.

Three support files make this work:

- **`intrinsics.rs`** — declares the LLVM intrinsics the math opcodes lower to (`llvm.exp.f64`, `llvm.log10.f64`, `llvm.fabs.f64`, `llvm.ceil.f64`, ...). *An opcode whose intrinsic isn't declared here crashes at emit time* — this is the recurring "register the intrinsic" class of fixes.
- **`declarations.rs`** — declares the function signatures (the `eval/setup/load` functions and the callback prototypes).
- **`callbacks.rs`** — emits the calls to simulator/runtime callbacks (`noise`, `$strobe`, file I/O) that the MIR left as opaque `Calls`.
- **`types.rs`** — maps the handful of scalar types (`f64`, `i32`, `bool`, `pointer`) to LLVM types.

12.4 What functions get generated

Per module, `codegen` produces the native implementations of the OSDI entry points (Chapter 13): the parameter-setup functions, the `eval` function (residual + Jacobian for the present inputs), and the

load/noise functions that stamp results into the simulator's matrix. LLVM optimises each (-O level from the CLI), and emits an object file (.o). You can inspect the IR at either stage:

```
openvaf-r --dump-unopt-ir resistor.va # LLVM IR straight out of MIR→LLVM
openvaf-r --dump-ir      resistor.va  # after LLVM's own optimisation
```


The object files are handed back to **compile()**, which links them (Chapter 13) into the final **.osdi**.

Chapter 13 — Stage 9: the OSDI descriptor and linking

Crates: `openvaf/osdi` (descriptor + data layout + entry-point codegen), `openvaf/linker` (final link). This is where a compiled module becomes a *loadable device* with a stable ABI.

13.1 What the simulator needs from the **.osdi** file

A circuit simulator that has never heard of your model needs a lot of metadata before it can use it: how many terminals, what parameters (name, type, default, bounds, units), how much memory to allocate per model card and per device instance, where each field sits in that memory, and function pointers to call for setup, evaluation, and matrix loading. All of that is the OSDI descriptor — a C struct the **.osdi** exports, one per module. Its shape (`openvaf/osdi/src/metadata.rs`, and the C headers in `openvaf/osdi/header/`) includes, among many fields:

```
// conceptual shape of the per-module OSDI descriptor
uint32_t num_nodes;           // DAE unknowns (terminals + internal nodes)
uint32_t num_terminals;       // externally-visible ports
uint32_t num_params;          // model + instance parameters
uint32_t instance_size;       // bytes to allocate per device instance
uint32_t model_size;          // bytes to allocate per model card
OsdiParamOpvar* param_opvar;  // parameter/opvar metadata (names, types, defaults,
    ↪ bounds)
OsdNode* nodes;               // node names & disciplines
OsdJacobianEntry* jacobian_entries; // which (row,col) matrix entries this device
    ↪ stamps
uint32_t node_mapping_offset, jacobian_ptr_resist_offset, ...; // where things live
    ↪ in instance data

// function pointers the simulator calls:
void* (*setup_model) (...);
void* (*setup_instance)(...);
uint32_t (*eval) (...); // compute residual + Jacobian for present inputs
void (*load_residual_resist)(...); void (*load_residual_react)(...);
void (*load_jacobian_resist)(...); void (*load_jacobian_react)(...); void
    ↪ (*load_jacobian_tran)(...);
void (*load_noise) (...);
// ... plus small-signal / AC-stim / limit / bound-step entry points ...
```

13.2 Instance data and model data: the memory layout

The single most important thing `osdi` computes is the memory layout of two blocks the simulator allocates and hands back on every call:

- Model data — one per **.model** card. Holds the model-level parameters (those *not* marked `type="instance"`), the "given" flags (which parameters the netlist actually set), and cached parameter-derived values.
- Instance data — one per device instance. Holds the instance parameters, the operating-point variables (opvars), the **node_mapping** (which global matrix row each local node maps to, after node collapse), the Jacobian pointers (one `double*` per matrix entry, pointing straight into the simulator's sparse matrix), persistent state slots (for `ddt/idt/edge` detectors), and the `eval` output cache.

The layout is described in the project's own `internals.md`; the key ideas:

Instance data:

```
given_flags[]      : one bit per instance parameter – "was it set by the netlist?"
jacobian_ptr[]     : one double* per resistive Jacobian entry (into the sim's matrix)
jacobian_ptr_react[]: one double* per reactive Jacobian entry
node_mapping[]     : one u32 per unknown – its row in the global system
collapsed[]        : one i8 per collapsible node pair – did it collapse?
state_idx[]        : persistent-state indices (for limiting/idt/ddt)
<parameters>      : the instance parameters and opvars
<cache>            : intermediate results & non-trivial Jacobian contributions
eval_outputs[]     : opvars, residuals, bound_step (deduplicated, order not guaranteed)
```

Scalars are laid out by type: integer→i32, real→f64, string→pointer, and vectors are contiguous arrays of those with a statically-known length. The given flags are packed 32 to a u32 word so the simulator can quickly ask "did the user set parameter *k*?" via the exported `given_flag_instance(inst, id) / given_flag_model(model, id)` functions — important because CMC models behave differently when a parameter was defaulted vs. explicitly set.

The reason the compiler, not the simulator, decides this layout is that only the compiler knows the model's internals — how many internal nodes, how many state slots, which Jacobian entries survive sparsification. It computes byte offsets using LLVM's own `LLVMOffsetOfElement` on the generated struct types, so the descriptor's offsets exactly match the machine code's field accesses.

13.3 The entry-point functions

`osdi` generates (as LLVM IR, via `mir_llvm`) the native bodies behind those function pointers. The files map one-to-one to the API:

- **setup.rs** → `setup_model / setup_instance`: apply parameter defaults, run range checks, pre-compute parameter-only quantities, and resolve node collapse for this instance.
- **eval.rs** → `eval`: the hot function. Given the present inputs (read out of instance/model data), run the optimised residual+Jacobian MIR and write results into the `eval_outputs` cache. It is guarded by flag bits the simulator passes — e.g. compute DC only, include reactive terms, this is the `@(initial_step) / @(final_step)` call, this is an AC/limit call — so one function serves every analysis mode.
- **load.rs** → `load_residual_* / load_jacobian_* / load_noise`: take the cached `eval` results and *stamp* them into the simulator's RHS vector and matrix through the `jacobian_ptr[] / node_mapping[]` indirection. Splitting `eval` (compute) from `load` (stamp) lets the simulator evaluate once and stamp into different structures (e.g. for harmonic balance, using the offset-based loaders from `internals.md`).
- **noise.rs** → the noise spectra (white/flicker/table/ac-stim) for `.noise` analysis.
- **access.rs** → the `given_flag_*` accessors and parameter get/set helpers.

13.4 Strings, natures, disciplines

Model text — parameter names, node names, units, `$strobe` format strings — is interned into a string table (`Rodeo/literals` in `osdi::compile`) and emitted once. Natures, disciplines, and their attributes (units, tolerances, `ddt/idt` nature links) get their own descriptor arrays (`nda_arrays`), so a host that cares about physical units can read them.

13.5 Emitting object files and linking (**linker**)

`osdi::compile` emits several LLVM modules per device (it pre-allocates `modules.len() * 4` object-file slots — `setup`, `eval`, `load`, and descriptor code roughly), optimises each at the requested `-O` level, and

writes native `.o` files. Back in `compile()` (Chapter 2), the linker crate links them all into one shared library:

```
link(None, &opts.target, lib_file.as_ref(), |linker| {
    for path in &paths { linker.add_object(path); }
})?;
```

The linker crate abstracts over the platform linker (`ld/lld/link.exe`) and target so the same code produces a `.so`, `.dylib`, or `.dll` — renamed `.osdi` by convention. The intermediate `.o` files are then deleted. The result is a single, self-contained, ABI-stable device library.

13.6 The version handshake

The descriptor carries the OSDI ABI version. This repository's compiler emits the reloaded descriptor (base 0.4, extended through 0.7), and a host negotiates against it. If you compile with a stale binary that only writes v0.4 base entry points and load it into a host expecting the ≥ 0.7 layout, the host rejects it with a clear version error — the exact failure you hit when mixing compiler and simulator generations. The extensions between 0.4 and 0.7 are additive descriptor fields (a longer descriptor whose prefix stays 0.3/0.4-compatible, which is why **OSDI_DESCRIPTOR_SIZE** is exported for hosts that stride the descriptor array manually).

Chapter 14 — A complete worked example: **resistor.va** end to end

Let's trace one tiny model through every stage, using the compiler's own dump flags so every listing below is *real output*, not a sketch. The model:

```
`include "disciplines.vams"
module resistor(p, n);
    inout p, n;
    electrical p, n;
    parameter real R = 1000.0 from (0:inf);    // R > 0
    analog begin
        I(p, n) <+ V(p, n) / R;                // Ohm's law
    end
endmodule
```

14.1 Front end (Stages 1–3)

Reading, preprocessing, parsing, name resolution and type checking establish: two electrical nodes `p, n`; one branch `(p, n)`; one real parameter `R` constrained to `(0, ∞)`; and one contribution statement whose left side is the flow `I(p, n)` and whose right side is a real expression `V(p, n)/R`. No type errors. (One *lint* does fire — see §14.6.)

14.2 Lowering to MIR (Stage 4a) — the residual only

`openvaf-r --dump-unopt-mir resistor.va` prints the analog block as SSA, before any DAE or derivative work:

```
Unoptimized MIR (no DAE) of resistor
function %(v16, v17, v19) {
```

```

block2:
  jmp block0
block0:
  v18 = fdiv v16, v17      ; v18 = V / R
  v20 = optbarrier v18     ; fence the output value
  jmp block1
block1:
}

```

Three things to read here:

- The function's *parameters* are v16, v17, v19 — these are the ParamKind inputs from Chapter 7. The eval interner (--dump-mir) labels them:

Parameters:

```

Voltage { hi: node0, lo: Some(node1) } .. V("p","n") -> v16
Param(Parameter { id: ParamId(0) })    .. "R"         -> v17
Current(Unnamed { hi: node0, lo: node1 }) .. I("p","n") -> v19
ParamSysFun(mfactor)                   -> v21

```

So v16 is $V(p, n)$, v17 is R , and (once the DAE is built) v21 is $\$mfactor$, the device multiplier every model gets for free.

- The single real computation is $v18 = \text{fdiv } v16, v17$ — literally V/R . That is the resistive residual $I(x)$.
- $v20 = \text{optbarrier } v18$ fences the result so the optimiser can't dissolve it before the OSDI layer attaches it to the branch. The eval interner's *Outputs* confirm the wiring:

Outputs:

```

Contribute { dst: (p,n), reactive: false, voltage_src: false } -> v20

```

i.e. "the non-reactive current contribution to branch (p,n) is value v20". Exactly the `PlaceKind::Contribute` from Chapter 7.

14.3 DAE + automatic differentiation (Stages 4b, 6)

Building the DAE turns this into $I(x) + dQ/dt = 0$ with the two node voltages as unknowns, and asks the differentiator for $\partial I / \partial V$. The "partially optimized MIR (with DAE)" dump shows the derivatives appearing right next to the residual:

```

function %(v16, v17, v19, v21) {          ; v21 = $mfactor now present
  v6 = fconst 0x1.0p0                     ; the constant 1.0
  block2:
    v18 = fdiv v16, v17                    ; I = V/R          (residual)
    v24 = fmul v17, v17                    ; R*R              (quotient-rule denom)
    v25 = fdiv v6, v17                     ; 1/R = d(V/R)/dV  (the Jacobian entry!)
    v20 = optbarrier v18                   ; residual output
    v31 = fmul v21, v18                    ; mfactor * I
    ...
    v35 = fmul v21, v25                    ; mfactor * (1/R)   (scaled Jacobian)
    ...
}

```

Read that middle line as the whole reason OpenVAF exists: the model author wrote only V/R , and the compiler *derived* $\partial I / \partial V = 1/R$ symbolically as $v25 = \text{fdiv } 1.0, v17$. It also computed $\partial I / \partial R$ -style terms and the $\$mfactor$ scaling ($v31 = \text{mfactor} \cdot I$, $v35 = \text{mfactor} \cdot (1/R)$), because the simulator

stamps `mfactor`· everything to model m parallel devices. For a transistor these few lines would be thousands, all machine-generated the same way.

14.4 Optimisation (Stage 7)

Constant folding, GVN, inst-combine and dead-code elimination then prune the graph: unused fnegs and duplicate products vanish, the `1.0` constant is folded into the reciprocal, and only the values that reach a real output (the residual `v20` and the live Jacobian entries) survive. What's left is the minimal set of instructions needed to produce the residual and its non-zero Jacobian entries.

14.5 Codegen + OSDI (Stages 8–9)

`--dump-ir` shows the residual/Jacobian MIR turned into the native `eval` function. Its LLVM IR opens like this:

```
define noundef i32 @eval_0(ptr %model, ptr %inst, ptr %model_data, ptr %sim_info) {
    %5 = getelementptr %OsdiSimInfo, ptr %sim_info, i64 0, i32 5
    %6 = load i32, ptr %5                ; read the eval flags
    %7 = and i32 %6, 1                  ; is the "compute" bit set?
    br i1 (%7 == 0), label %skip, label %do_eval
do_eval:
    %10 = load double, ptr (inst field 8) ; read R out of instance data
    %12 = load ptr,    ptr (sim_info field 2) ; the solution vector
    %14 = load i32,    ptr (inst field 3) ; node_mapping[0] -> p's row
    %16 = getelementptr double, ptr %12, i64 sext(%14)
    %17 = load double, ptr %16            ; V(p) from the solution vector
    ...
}
```

This is Chapter 12 made concrete: the `ParamKind` inputs became loads — R from the instance-data struct, and $V(p)$ by indexing the simulator's solution vector through `node_mapping[]` (Chapter 13). The whole body is gated by an `and %flags, 1` so the one function serves every analysis mode. `osdi` then wraps this in the descriptor (`num_nodes`, `jacobian_ptr` offsets, the `eval/load/setup` pointers) and the linker produces:

```
resistor.osdi      # a 54 KB shared library, loadable by ngspice et al.
```

which is exactly what you'd `pre_osdi` into a simulator.

14.6 The diagnostic you'd actually see

Compiling the model above *does* print one warning, which is a nice illustration of the lint system (Chapter 15):

```
warning[L018]: module name 'resistor' collides with ngspice's built-in 'Resistor' device
= help:  `.model <name> resistor` may silently bind to ngspice's built-
in device instead of
        this OSDI module; rename 'resistor' to something that doesn't collide.
```

The compiler carried enough *semantic* knowledge (the module's name) and enough *domain* knowledge (a table of reserved simulator device names) to warn about a footgun that has nothing to do with the Verilog-A language itself — a good example of the value the HIR layer adds over raw parsing.

Chapter 15 — Cross-cutting machinery: database, diagnostics, tests

Three systems run *underneath* every stage. They aren't a pipeline step, but you can't understand the codebase without them.

15.1 The incremental database (**salsa**)

As introduced in §3.4, OpenVAF is organised as memoised queries over a **salsa** database, in two layers:

- **BaseDB** (`openvaf/basedb`) — the syntactic layer: the VFS, file texts, preprocessing, parsing, the AST-id map (stable ids for AST nodes so the semantic layer can refer to them across edits), line indexing (for turning byte offsets into line:col), and linting.
- **CompilationDB** (`openvaf/hir`) — the semantic layer built on top: item trees, name resolution, type inference, and the `compilation_unit()` façade that the driver talks to.

A query is just a function whose result **salsa** caches keyed on its inputs. `db.parse(file)`, `db.item_tree(file)`, `db.inference_result(body)` are queries; the first call computes and memoises, later calls return the cache. For the batch compiler the practical payoffs are laziness (nothing runs until an output demands it — creating the DB in `compile()` does no parsing; the first `collect_modules` query triggers it) and sharing (a header parsed once serves every module; a discipline resolved once serves every branch). The custom **salsa** fork the project pins is the same machinery rust-analyzer uses to stay responsive on huge codebases.

15.2 Diagnostics and lints (**basedb::diagnostics**, **basedb::lints**)

Every stage can emit diagnostics. A diagnostic carries a message, a severity, and one or more spans that (via the preprocessor source map, Chapter 4) point at real source — through include and macro layers — so the arrow lands on what the user typed. Diagnostics flow into a `DiagnosticSink`; the CLI uses a `ConsoleSink` that renders the familiar `error[...]` `--> file:line:col` blocks with source snippets and help: notes.

There are two grades:

- Hard errors — the program is ill-formed (unknown identifier, type mismatch, illegal contribution). `sink.summary()` returns "fatal", and `collect_modules` returns `None`, so `compile()` stops with `FatalDiagnostic` before any code is generated.
- Lints — legal but suspect code. Each lint has a name, a code (e.g. `L018 reserved_module_name` from §14.6), and a default level (`allow/warn/deny`) that the user can override on the CLI or with source attributes (`(* ... *)`). Lints are how OpenVAF flags domain footguns — a module named like a built-in device, a parameter defaulted outside its own declared range, a probably-unintended discontinuity — without rejecting the model.

This diagnostics-first design (lossless CST + source map + rich sinks) is why the compiler can report *several* problems per run with precise locations, instead of dying at the first.

15.3 Testing: snapshots and integration models

Two testing styles keep the compiler honest.

Snapshot (“expect”) tests. For a given input, the test captures a stage’s output as text and compares it to a committed reference file. `openvaf/test_data/` is organised by stage — `ast/`, `item_tree/`, `body/`, `contributions/`, `dae/`, `init/`, `mir/`, `osdi/`, plus `ui/` and `syn_ui/` for diagnostic text. When you *intentionally* change what a stage emits, you regenerate the references with `UPDATE_EXPECT=1 cargo test`; an *unintentional* change shows up as a failing diff. Some references are `.snap` files; a few (notably in `mir_autodiff`) are hard-coded in the test source and updated by hand. Because the MIR interpreter (`mir_interpret`, §11.3) can compute reference numbers independently of codegen, numeric regressions are caught even when the text of the IR is unchanged.

Integration tests. `integration_tests/` holds *real* industry models — BSIM3/4/6, BSIM-CMG/-IMG/-SOI, BSIMBULK, HICUM, MEXTRAM, EKV, PSP, ASMHMT, and small primitives (CCCS, VCVS, ...). These compile the actual `.va`, load the resulting `.osdi` through a mock OSDI host, and check that `setup/eval/load` behave. They are the safety net that catches whole-pipeline breakage — the kind a single-stage snapshot can’t — and they exercise the ABI against a simulator-like consumer. Some carry restrictive model licenses that don’t affect the compiler’s own license.

Fast tests run by default; **RUN_SLOW_TESTS=1** adds the heavy models. This layered test strategy (per-stage snapshots + end-to-end integration + an interpreter oracle) is what makes it feasible to add a language feature or fix a numeric law and be confident nothing else moved.

Chapter 16 — How a simulator uses the **.osdi** at runtime

The compiler's job ends when `resistor.osdi` is written, but you only understand *why* the descriptor looks the way it does by seeing how the simulator drives it. Here is the lifecycle, using ngspice as the example host.

16.1 Load and register

At `pre_osdi` `resistor.osdi` (or an autoload), the host `dlopen`s the library, checks the OSDI version in the descriptor (rejecting a mismatch, §13.6), and reads the descriptor array: it now knows the device `resistor` has 2 terminals, a real parameter `R` (default 1000, range $(0, \infty)$), how big its instance/model blocks are, and the addresses of `setup_model`, `setup_instance`, `eval`, the `load_*` functions, etc.

16.2 Build the circuit (once)

When the netlist instantiates the device (`R1 a b resistor R=2k`), the host:

1. allocates a model-data block for each `.model` card and calls **setup_model** — which applies parameter defaults, marks "given" flags for what the netlist set, runs range checks, and precomputes model-level quantities;
2. allocates an instance-data block per device and calls **setup_instance** — which does the same for instance parameters, resolves node collapse (does an internal node vanish for these parameters?), and fills `node_mapping[]` with the global matrix row of each of the device's unknowns;
3. wires the Jacobian pointers: for each `(row, col)` entry the device declared, the host stores, in the instance's `jacobian_ptr[]`, the address of that exact slot in its global sparse matrix. Now the device can stamp a derivative by writing through a pointer, with no lookup.

16.3 Solve (the hot loop)

For each analysis point (a DC operating point, each transient timestep, each AC frequency) the host runs Newton's method (§3.1). Each iteration, for each instance:

1. **eval(model, inst, model_data, sim_info)** — the function we watched get generated in §14.5. It reads the present unknown values out of the solution vector (through `node_mapping[]`) and the parameters out of instance/model data, runs the optimised residual+Jacobian code, and writes the results into the instance's `eval_outputs` cache. `sim_info` carries flag bits selecting what to compute (resistive only, include reactive, this is the initial/final step, AC, limiting) so one function covers every mode.
2. **load_residual_resist / load_residual_react** — add this device's I and Q contributions into the host's right-hand-side vector.
3. **load_jacobian_resist / load_jacobian_react / load_jacobian_tran** — add this device's $\partial I / \partial x$ and $\partial Q / \partial x$ into the matrix, through the pre-wired `jacobian_ptr[]`. (The `tran` variant folds in the d/dt factor for reactive entries.)

The host then solves the linear system for Δx , updates the guess, and repeats until the residual is negligible. The split between **eval** (compute once) and **load** (stamp) is what lets the host evaluate a device a single time and stamp its numbers into several places (e.g. real and imaginary matrices for AC, or many time-shifted copies for harmonic balance via the offset loaders in `internals.md`).

16.4 Small-signal, noise, and operating-point output

- AC / small-signal: the host uses the reactive Jacobian and any `ac_stim` sources (partitioned into their own descriptor slots, §13.1) to solve the linearised system at each frequency.
- Noise: `load_noise` returns each source's spectral density (white/flicker/table), which the host propagates to the output.
- Operating-point variables (opvars): quantities the model exposed with (`*desc="..."`) are read out of `eval_outputs` so the user can `print @device[gm]` etc.

16.5 Why this shape

Every design choice in the descriptor traces back to this loop being *hot*: parameters are pre-processed in **setup** so **eval** does no defaulting; Jacobian slots are pre-wired so **load** does no matrix search; **eval** and **load** are separate so compute isn't repeated per stamp target; the whole thing is native machine code with a flag-gated fast path. The compiler front end exists to *understand* the model; the back end and OSDI layer exist to make this loop as cheap as physically possible.

Chapter 17 — Reference appendices

17.1 Crate map

The `openvaf/` workspace, by role. (Utility crates live under `lib/`.)

Crate	Stage	Responsibility
<code>vfs</code>	1	Virtual file system; assigns <code>FileIds</code>
<code>lexer</code>	1	Byte-level tokeniser
<code>tokens</code>	1	Classifies raw tokens into <code>SyntaxKind</code> ; keyword table
<code>preprocessor</code>	1	<code>`include/`define/`ifdef</code> ; macro expansion; source map
<code>parser</code>	2	Recursive-descent + Pratt parser; emits tree events
<code>syntax</code>	2	Rowan CST + typed AST overlay
<code>basedb</code>	infra	Syntactic salsa DB: files, parsing, lints, line index, AST-id map
<code>hir_def</code>	3	Item tree, name resolution, built-in registry, paths
<code>hir_ty</code>	3	Type inference & checking, validation
<code>hir</code>	3/infra	CompilationDB: the semantic DB façade
<code>hir_lower</code>	4	HIR $\rightarrow$ MIR lowering; the <code>HirInterner</code> (<code>ParamKind/PlaceKind</code>)
<code>mir</code>	5	The SSA IR: Function, DFG, opcodes, layout
<code>mir_build</code>	5	SSA construction (variables $\rightarrow$ values + φ)
<code>sim_back</code>	4/9	Topology, DAE system, node collapse, <code>CompiledModule</code> , <code>collect_modules</code>
<code>mir_autodiff</code>	6	Automatic differentiation $\rightarrow$ Jacobian
<code>mir_opt</code>	7	SCCP, GVN, inst-combine, DCE, CFG simplification
<code>mir_interpret</code>	7/test	MIR evaluator: constant folding + test oracle
<code>mir_llvm</code>	8	MIR $\rightarrow$ LLVM IR; intrinsics, declarations, callbacks
<code>target</code>	8	Target-machine / data-layout configuration
<code>osdi</code>	9	OSDI descriptor, instance/model data layout, entry-point codegen
<code>linker</code>	9	Links object files into the <code>.osdi</code> shared library
<code>openvaf</code>	driver	The <code>compile()/expand()</code> library API
<code>openvaf-driver</code>	driver	CLI parsing (<code>cli_def/cli_process</code>), main, crash reports

Crate	Stage	Responsibility
<code>mir_reader</code>	test	Parses textual MIR (for tests/debugging)

17.2 MIR opcode quick reference

Prefixes: I=integer, F=float(real), B=bool, S=string.

```
Unary/cast : Inot Bnot Fneg Ineg  Ficast IFcast BICast IBcast FBcast BFCast OptBarrier
Real math1 : Sqrt Exp Ln Log Clog2 Floor Ceil
              Sin Cos Tan Asin Acos Atan Sinh Cosh Tanh Asinh Acosh Atanh
Int arith  : Iadd Isub Imul Idiv Irem  Ishl Ishr Iashr  Ixor Iand Ior
Real arith : Fadd Fsub Fmul Fdiv Frem
Compare   : Ilt Igt Ige Ile   Flt Fgt Fge Fle
              Ieq Feq Seq Beq   Ine Fne Sne Bne
Real math2 : Hypot Atan2 Pow
Control    : Br  Jmp Exit Call Phi
```

Every opcode has fixed operand/result types (the MIR is untyped by *values*, typed by *opcode*). Casts are explicit; there is no implicit int $\leftrightarrow$ real coercion inside MIR.

17.3 The two enums to memorise

The HIR $\leftrightarrow$ numeric boundary is these two (`openvaf/hir_lower/src/lib.rs`):

- **ParamKind** — an *input* the simulator supplies: `Voltage{hi,lo}`, `Current(..)`, `Temperature`, `Abstime`, `Param(..)`, `ParamGiven{..}`, `PortConnected{..}`, `ParamSysFun(..)`, `HiddenState(..)`, `ImplicitUnknown(..)`, `IsInitialStep`, `IsFinalStep`, `EventState(i)`, `EnableIntegration`, `EnableLim`, `PrevState/NewState`.
- **PlaceKind** — an *output* the simulator reads: `Contribute{dst,reactive,voltage_src}`, `Var(..)`, `FunctionReturn(..)`, `ImplicitResidual{..}`, `Param(..)/ParamMin/ParamMax`, `BoundStep`, `EventState(i)`, `AbsDelayTime(i)`, `LastCrossingDirection(i)`.

If you internalise “the eval function is `outputs = f(params)`, where `ParamKind` names the params and `PlaceKind` names the outputs, and `Contribute{reactive:false}/{reactive:true}` are the I and Q of $I + dQ/dt = 0$ ”, you understand the front-end/back-end handoff.

17.4 Debugging & inspection flags

```
openvaf-r --dump-unopt-mir model.va # MIR straight after lowering (no DAE)
openvaf-r --dump-mir       model.va # MIR after DAE build, autodiff, optimisation
openvaf-r --dump-unopt-ir  model.va # LLVM IR straight from MIR $\rightarrow$ LLVM
openvaf-r --dump-ir       model.va  # LLVM IR after LLVM optimisation
openvaf-r --help          # all flags (defines -D, include -I, opt
   $\hookrightarrow$  level, lints)
```

Environment switches used by maintainers:

```
OPENVAF_DAE_DEBUG=1 # print the DAE unknowns/residuals/Jacobian/noise as they're
   $\hookrightarrow$  built
RAYON_NUM_THREADS=1 # (debug builds) single-thread for step-through debugging
UPDATE_EXPECT=1 cargo test # regenerate snapshot references after an intended
   $\hookrightarrow$  change
RUN_SLOW_TESTS=1 cargo test # include the heavy real-model integration tests
```

17.5 Recipe: adding a Verilog-A built-in

Because so many language features are "a new built-in", the mechanical path is worth spelling out (it is the same one used to add `$clog2`, `$rtoi/$itor`, `$fgetc`, string relational ops, etc.):

1. **openvaf/syntax/src/name.rs** — register the spelling in the system-function/keyword tables so the lexer/name layer recognises it.
2. **openvaf/hir_def/src/builtin.rs** — add a `BuiltIn` enum variant and register it.
3. **openvaf/hir_ty/src/builtin.rs** (+ generated `builtin/generated.rs`) — give it a signature (argument requirements and result type). The array index in `generated.rs` must equal the enum discriminant.
4. **openvaf/hir_lower/src/expr.rs** — lower the call: emit the MIR opcode(s), or a callback, that implement it.
5. If it needs a runtime service or an LLVM intrinsic: register the intrinsic in **openvaf/mir_llvm/src/intrinsics** and/or add a callback in **openvaf/hir_lower/src/callbacks.rs** + **openvaf/osdi** (with a `stdlib` implementation for file/string ops).
6. If it's constant-foldable, teach **mir_interpret/const_eval** the same operation so compile-time evaluation and the test oracle agree.
7. Add snapshot + (ideally) a numeric example, and run the full test suite.

A recurring lesson: a Signature index is *operator-local*, so two operators can each have a Signature(2) meaning different things — always match on the operator too when you branch on a signature, or you get subtle cross-talk that only a full regression (not a targeted test) reveals.

17.6 Glossary

- AST / CST — Abstract / Concrete Syntax Tree. The CST is lossless (keeps whitespace/comments/errors); the AST is a typed view over it.
- Autodiff (AD) — computing exact derivatives by mechanically applying the chain rule to a program's operations.
- Basic block — a straight-line run of instructions with a single entry and a single exit (a branch/jump/return).
- Branch (Verilog-A) — a path between two nodes across which a potential (V) and through which a flow (I) are defined.
- DAE — Differential-Algebraic Equation system; here $I(x) + d/dt Q(x) = 0$.
- Discipline / Nature — Verilog-A's typing of physical quantities (electrical, thermal...) and their potential/flow natures (volts/amps...) with tolerances and units.
- DFG — Data-Flow Graph; the MIR's store of instructions/values/blocks and how they connect.
- Given flag — a per-parameter bit recording whether the netlist explicitly set it (vs. taking the default).
- HIR / MIR — High-level / Mid-level Intermediate Representation. HIR = meaning; MIR = numeric SSA.
- Jacobian — the matrix of partial derivatives of the residuals with respect to the unknowns; what Newton's method needs.
- Newton's method — iterative solver for non-linear systems: linearise, solve for a correction, repeat.
- Node collapse — merging two unknowns when parameters make an internal node redundant (e.g. a zero series resistance).
- Opvar — operating-point variable; a model quantity exposed for readout (`(*desc*)`).
- OSDI — Open Source Device Interface; the ABI a `.osdi` exports to any host simulator.
- Reactive / resistive — the Q (charge, differentiated in time) and I (instantaneous current) parts of a contribution.
- Residual — how far an equation is from being satisfied; the thing Newton drives to zero.
- salsa — the incremental query/memoisation framework the compiler's databases are built on.
- SSA — Static Single Assignment; each value is assigned once, merges use φ nodes.

- **\$mfactor** — device multiplier; simulate m identical devices in parallel by scaling contributions.
-

Closing note

OpenVAF-r is, at heart, a machine for answering one question — “*given these terminal voltages, what current does this device push, and how does that current change if I nudge each voltage?*” — for arbitrarily complicated physics, exactly and fast. Everything in this document serves that: the front end (Chapters 4–6) works out what the model *means*; lowering and `sim_back` (Chapters 7–9) turn its meaning into the DAE $I + dQ/dt = 0$; autodiff (Chapter 10) manufactures the Jacobian the author never wrote; optimisation and LLVM (Chapters 11–12) make it small and native; and the OSDI layer (Chapter 13) packages it behind a stable ABI so any simulator can run it in its Newton loop (Chapter 16). The resistor of Chapter 14 needed one line of physics and got a complete, differentiated, compiled device out the other end — and the very same pipeline scales, unchanged, to a ten-thousand-line BSIM-CMG.